

|                             |               |                                                |
|-----------------------------|---------------|------------------------------------------------|
| Student Name                | B. VARSHINI   |                                                |
| Student Registration Number | AUU23EGCSE065 | Class & Section: CSC-C                         |
| Study Level: UG/PG          | UG            | Year & Term: 1 <sup>ST</sup> & 3 <sup>RD</sup> |
| Subject Name                | PHYSICS-II    |                                                |

|                        |                        |
|------------------------|------------------------|
| Name of the Assessment | PROJECT IMPLEMENTATION |
| Date of Submission     | 21/05/24               |

## Real-Time Heart Rate Monitoring System Implementation

### Introduction

Heart rate monitoring is crucial in both healthcare and fitness domains as it provides essential insights into an individual's cardiovascular health. This project aims to create a real-time heart rate monitoring system using a pulse sensor, an Arduino microcontroller, and Processing software for data visualization. The system will read heart rate data from the pulse sensor, process it using the Arduino, and display the results graphically using Processing.

### Hardware Setup

#### Components Required:

- Arduino Uno:**
  - A microcontroller board used to read and process sensor data.
- SEN-11574 Pulse Sensor:**
  - An optical heart rate sensor that detects pulse signals from the fingertip.
- Jumper Wires:**
  - Wires used to establish electrical connections between the sensor and the Arduino.
- USB Cable:**
  - A cable to connect the Arduino to a computer for programming and power supply.
- Computer:**
  - A device equipped with Arduino IDE and Processing software for coding and visualization.

#### Connect the Pulse Sensor to Arduino:

- GND Connection:**

- Connect the GND pin of the pulse sensor to a GND pin on the Arduino to complete the circuit ground.
2. **VCC Connection:**
    - Connect the VCC pin of the pulse sensor to the 5V pin on the Arduino to provide power.
  3. **Signal Connection:**
    - Connect the signal pin (A0) of the pulse sensor to the A0 analog input pin on the Arduino to read sensor data.

### **Assemble the Hardware:**

1. **Sensor Placement:**
  - Secure the pulse sensor on a fingertip using a rubber band or tape. This ensures the sensor remains in place for accurate readings.
2. **Arduino Connection:**
  - Connect the Arduino to the computer using the USB cable. This not only powers the Arduino but also allows for programming and data transfer.

### **Software Development**

#### **Arduino Sketch:**

1. **Setup Arduino IDE:**
  - Open the Arduino IDE on your computer.
  - Create a new sketch by navigating to **File > New**.
2. **Import PulseSensor Playground Library:**
  - Install the PulseSensor Playground library by navigating to **Sketch > Include Library > Manage Libraries**, searching for "PulseSensor Playground", and clicking install.
3. **Arduino Code:**
  - Write the following code to read data from the pulse sensor, calculate heart rate, and send it to the serial port:

```
#include <PulseSensorPlayground.h>
```

```
const int PulseWire = A0;  
const int LED13 = 13;  
int Threshold = 550;
```

```
PulseSensorPlayground pulseSensor;
```

```
void setup() {
```

```
Serial.begin(9600);
pulseSensor.analogInput(PulseWire);
pulseSensor.blinkOnPulse(LED13);
pulseSensor.setThreshold(Threshold);

if (pulseSensor.begin()) {
  Serial.println("Pulse sensor initialized successfully!");
}
}

void loop() {
  int myBPM = pulseSensor.getBeatsPerMinute();
  int myIBI = pulseSensor.getInterBeatInterval();

  if (pulseSensor.sawStartOfBeat()) {
    Serial.print("BPM: ");
    Serial.print(myBPM);
    Serial.print(", IBI: ");
    Serial.println(myIBI);
  }
}
```

### **Processing Sketch:**

#### **1. Setup Processing Software:**

- Open Processing on your computer.
- Create a new sketch by navigating to **File > New**.

#### **2. Processing Code:**

- Write the following code to establish serial communication with Arduino and receive heart rate data:

```
import processing.serial.*;
```

```
Serial myPort;
String inString;
int BPM;
int IBI;
```

```
void setup() {
  size(800, 400);
```

```
myPort = new Serial(this, Serial.list()[0], 9600);  
myPort.bufferUntil('\n');  
}
```

```
void draw() {  
  background(255);  
  textSize(24);  
  fill(0);  
  text("Heart Rate Monitoring", 20, 40);  
  text("BPM: " + BPM, 20, 80);  
  text("IBI: " + IBI, 20, 120);  
}
```

```
void serialEvent(Serial myPort) {  
  inString = myPort.readStringUntil('\n');  
  if (inString != null) {  
    String[] values = split(inString, ',');  
    if (values.length > 1) {  
      BPM = int(trim(values[0].substring(5)));  
      IBI = int(trim(values[1].substring(5)));  
    }  
  }  
}
```

## Code Overview

### Arduino Sketch:

- **Initialization:**
  - The code initializes the pulse sensor and sets up serial communication.
  - The sensor reads analog data and blinks an LED on each detected heartbeat, providing a visual indication of the pulse detection.
- **Main Loop:**
  - The loop continuously reads heart rate and interbeat interval (IBI) values.
  - The data is formatted and sent to the serial port for further processing or visualization.

### Processing Sketch:

- **Setup:**

- Establishes serial communication with the Arduino to receive heart rate data.
- Sets up the graphical display window for visualizing the data.
- **Draw Loop:**
  - Continuously updates the display with the latest BPM and IBI values.
  - Uses text and graphical elements to represent heart rate data visually, providing a clear and immediate understanding of the user's heart rate status.
- **Serial Event:**
  - Reads data from the Arduino serial port and parses it to extract BPM and IBI values.
  - Updates the visual display based on the latest readings, ensuring real-time accuracy.

### **Running the System**

#### **Upload Arduino Sketch:**

##### **1. Compile and Upload:**

- Compile the Arduino sketch by clicking the checkmark icon in the Arduino IDE.
- Upload the sketch to the Arduino by clicking the right arrow icon.
- Open the serial monitor in the Arduino IDE to verify that heart rate data is being displayed correctly.

#### **Run Processing Sketch:**

##### **1. Start Processing:**

- Run the Processing sketch by clicking the play icon in the Processing IDE.
- Verify that the Processing window displays real-time visualizations of heart rate data, including BPM and IBI values.

### **Testing Procedures**

#### **Sensor Placement Test:**

##### **1. Verify Placement:**

- Ensure the pulse sensor is securely attached to the finger to avoid signal loss or interference.
- Confirm that the sensor detects heart rate accurately without interruptions.

#### **Data Accuracy Test:**

##### **1. Manual Verification:**

- Compare the heart rate readings from the system with manual pulse counting to verify accuracy.
- Ensure the system provides consistent BPM and IBI measurements, confirming its reliability.

### **Interactivity**

#### **User Interaction:**

##### **1. Interactive Features:**

- Enhance the Processing sketch with interactive features such as zooming, panning, or adjusting visualization parameters.
- Allow users to customize display settings according to their preferences, improving user experience and engagement.

#### **Real-Time Updates:**

##### **1. System Responsiveness:**

- Test the system's responsiveness by varying the heart rate through physical activities such as light exercise or deep breathing.
- Observe real-time updates in the visualizations, demonstrating the system's capability to handle dynamic changes in heart rate.

### **Troubleshooting**

#### **Serial Communication Issues:**

##### **1. Check Settings:**

- Verify the serial port settings in both Arduino and Processing sketches.
- Ensure the correct port is selected and that baud rates match to establish a stable connection.

#### **Hardware Malfunction:**

##### **1. Verify Connections:**

- Inspect the hardware connections between the pulse sensor and Arduino to ensure they are secure and correct.
- Replace any faulty components if necessary to maintain system integrity.

#### **Data Parsing Errors:**

##### **1. Debugging:**

- Debug the Processing sketch to ensure proper parsing of incoming data, identifying and fixing any issues in the code.
- Implement error handling to manage exceptions gracefully, preventing crashes or application freezes.

### **Conclusion**

By following this detailed implementation guide, you can successfully create a real-time heart rate monitoring system using a pulse sensor, Arduino, and Processing software. This project demonstrates the integration of hardware and software components to provide valuable insights into cardiovascular health in real-time. Proper testing and troubleshooting will ensure the system's accuracy and reliability for various applications, including healthcare, fitness tracking, and medical research.